

AI辅助技术穿刺代码实战总结

AI辅助技术穿刺代码实战总结

一、背景与挑战

当前产品配置器在反向推理及最优配置推荐方面存在能力不足。前期通过初步技术洞察，发现约束求解可能是解决方案之一，项目组决定采用实际业务场景进行技术打样，完成可运行的Demo开发，包括：

- 基于约束模型的产品配置业务建模构建
- 基于约束编程完成X产品配置逻辑的代码开发
- 基于约束模型的产品配置求解器（仅参数反向推理）的开发

约束满足问题（Constraint Satisfaction Problems, CSP）定义一组对象（变量），其状态必须满足许多约束或限制，广泛应用于规划调度、CAD等领域。其核心包括约束的表示（[约束编程](#)）和约束求解器，是学术界研究的热点之一（[2021年理论计算机最高荣誉"哥德尔奖"](#)就是研究该领域的获得）。

约束编程是一种编程范式（思维方式），它让你描述问题是什么，而不是规定计算机如何一步步解决问题。对于熟悉面向对象编程（如Java）的开发者来说，学习约束编程的跨度很大（难度远大于从Java到Python）。另外，加上求解器这个黑盒，涉及到复杂的算法，遇到问题时定位复杂。

团队没有这方面基础，仅有的2名开发人员具有Java背景，项目时间约2个月，需要完成准商用版本，时间紧、难度大。

二、成果和与AI协作经验总结

成果

首次在项目组引入约束求解技术，完成了一个实际产品从配置业务建模、产品配置算法开发、测试、发布到应用的全流程打通（无UI），2个月完成项穿刺（不使用AI，预计3~6个月完成），代码方面的关键数据

- **总代码量**: Java代码26KLoc(其中5K过程验证代码)，代码开发1.5人，80%代码AI输出
- **冲突检查功能**: 不使用AI辅助，1个人开发2周；使用AI，1个人2小时完成
- **编程规范修改**: 7000个违规项，使用AI约3天完成

AI协助经验

大模型虽然知识渊博，但有时也会不靠谱，甚至一本正经地给出错误答案。在技术预研项目中（完全不熟悉的领域），如何与AI有效协作，快速高效地完成项目目标，是一个值得深入探讨的实战问题。

本文基于上述项目实践，总结在不同项目阶段与AI协作的代码实战经验：

阶段	角色-AI	角色-人	协同要点
初期阶段	博学的AI导师	好学的学生	不懂就问、触类旁通
中期阶段	专业的AI教授	务实业务专家	同频共振，深度协作
后期阶段	勤劳的AI高级程序员	需求分析&测试人员	规范输出，保障质量

三、AI辅助代码实战详情

初期阶段：博学的AI导师 VS 好学的学生 不懂就问、触类旁通

在项目初期，团队成员对约束求解技术完全陌生，充分利用大模型博学的特点，补充知识的盲点，此时AI的作用类似于导师，但需要注意：

- **如何高效学习：**采用关联学习法，基于自己已有的知识，快速学习，下面截图显示的通过自己熟悉的java来类别学习约束编程的语法

约束编程中是如何表达Java高级语言中的场景语法（如：赋值语句），请场景，逐个说明，按下列要求输出

一、输出的使用java库位：ortools-java 9.12.4544

二、输出的格式为：

场景1: xxx (如：变量与约束的直接映射)

和java的相同点和不同点： xxx (如：都有变量，都有常量)

案例： xxx (如: $x = y + 5$)

MiniZinc约束表达:xxx (如:constraint $x = y + 5$;))

Java高级语言表达： xxx， 如：

```
int x=1;
int y=2;
x = y+ 5;
```

三、java输出的代码分两部分

//依赖的包:

//每个例子包装为一个函数

```
public static void testCase1_xxx(){
    //xxxx

}
```

场景2: 条件赋值 (If-Then-Else)

和Java的相同点和不同点

- 相同: 都能表达条件逻辑
- 不同: CP中使用布尔变量和蕴含约束, 而非流程控制语句

案例

```
if (a > b) then x = 10 else x = 5
```

MiniZinc约束表达

javascript

复制

下载

```
constraint (a > b) -> (x == 10) /\ (a <= b) -> (x == 5);
```

Java高级语言表达

java

复制

下载

```
int a=7, b=3;  
int x = (a > b) ? 10 : 5; // 条件运算 (x=10)
```

ORTools约束表达

java

复制

下载

```
public static void testCase2_ConditionalAssignment() {  
    CpModel model = new CpModel();  
    IntVar a = model.newIntVar(0, 10, "a");  
    IntVar b = model.newIntVar(0, 10, "b");  
    IntVar x = model.newIntVar(0, 10, "x");  
  
    // 创建条件布尔变量  
    BoolVar cond = model.newBoolVar("cond");  
    model.addGreaterThanOrEqual(a, b).onlyEnforceIf(cond);  
    model.addLessOrEqual(a, b).onlyEnforceIf(cond.not());  
  
    // 条件赋值  
    model.addEquality(x, 10).onlyEnforceIf(cond);  
    model.addEquality(x, 5).onlyEnforceIf(cond.not());  
  
    System.out.println("约束: if(a>b) x=10 else x=5");  
}
```

- 大模型输出很多，基于关键概念，逐步建立当前这个知识点的知识网

场景2: 条件赋值 (If-Then-Else)

和Java的相同点和不同点

- 相同：都能表达条件逻辑
- 不同：CP中使用布尔变量和蕴含约束，而非流程控制语句

案例

```
if (a > b) then x = 10 else x = 5
```

MiniZinc约束表达

javascript

复制 下载

```
constraint (a > b) -> (x == 10) /\ (a <= b) -> (x == 5);
```

Java高级语言表达

java

复制 下载

```
int a=7, b=3;  
int x = (a > b) ? 10 : 5; // 条件运算 (x=10)
```

ORTools约束表达

java

复制 下载

```
public static void testCase2_ConditionalAssignment() {  
    CpModel model = new CpModel();  
    IntVar a = model.newIntVar(0, 10, "a");  
    IntVar b = model.newIntVar(0, 10, "b");  
    IntVar x = model.newIntVar(0, 10, "x");  
  
    // 创建条件布尔变量  
    BoolVar cond = model.newBoolVar("cond");  
    model.addGreaterThanOrEqual(a, b).onlyEnforceIf(cond);  
    model.addLessOrEqual(a, b).onlyEnforceIf(cond.not());  
  
    // 条件赋值  
    model.addEquality(x, 10).onlyEnforceIf(cond);  
    model.addEquality(x, 5).onlyEnforceIf(cond.not());  
  
    System.out.println("约束: if(a>b) x=10 else x=5");  
}
```

- **AI偶尔不太靠谱**：对AI提供的代码和方案，必须实际运行验证（see is cheap, run the code），通过读代码，调代码深入理解其原理，也建议大模型给的答案是否正确

下面的例子就是生成的代码不可运行（使用不存在的api等，版本不匹配等）

```
src > main > java > J ConfigServerSimple.java > ConfigServerSimple > main(String[])

8  public class ConfigServerSimple {
9      public static void main(String[] args) {
29
30          // 计算总内存容量和总内存条数
31          IntVar totalMemCapacity = model.newIntVar(lb: 0, ub: 512, name: "total_mem_capacity");
32          model.addEquality(totalMemCapacity,
33              LinearExpr.weightedSum(new IntVar[]{mem16Count, mem32Count}, new long[]{16, 32}));
34
35          IntVar totalMemBars = model.newIntVar(lb: 0, ub: 32, name: "total_mem_bars");
36          model.addEquality(totalMemBars,
37              LinearExpr.sum(new IntVar[]{mem16Count, mem32Count}));
38
39          // ===== 添加约束条件 =====
40
41          // 约束1: 使用场景约束 (办公场景要求CPU频率≥2.6GHz)
42          IntVar isWork = model.newBoolVar(name: "is_work");
43          model.addEquality(scenario, value: 0).onlyEnforceIf(isWork);
44          model.addNotEqual(scenario, 0).onlyEnforceIf(isWork.not());
45          model.addEquality(cpuChoice, value: 1).onlyEnforceIf(isWork); // 只有cpu_002满足2.6GHz
46      }
```

中期阶段：专业的AI教授 VS 务实业务专家，同频共振，深度协作

到项目中期，对约束求解技术有了基本了解后，与AI的协作进入新阶段：

- **同频共振**：能够提出专业的问题，获得专业的回答，通过不断学习得到线索，下面案例是使用大模型辅助配置器建模，针对提供方法，进一步深入探讨（可选性很多），这时感觉AI—

当前的ParaVar.var定义为IntVar，如果是String类型参数或多选型参数，应该名字建模呢，请参考一下SAP Variant、ConfigIT、Tacton等配置器的建模方式给出答案，结合案例来说明



已深度思考 (用时 18 秒)

我主要参考了行业常见的配置器实践（如SAP Variant Configuration ⁴、Tacton CPQ ³ 以及其他产品配置管理概念 ⁷），为你提供 String 及多选型参数的建模思路和基于 OR-Tools 的实现示例。

参数类型扩展建模方案

参数类型	存储变量类型	选项选择变量	特点
整数型 (Int)	IntVar	Map<Integer, BoolVar> (选项ID → 是否选中)	选项
字符串型 (String)	IntVar (编码值)	Map<Integer, BoolVar> (选项ID → 是否选中)	需维码
多选型 (Multi-Select)	IntVar[] (位掩码) 或 BoolVar[]	Map<Integer, BoolVar> (选项ID → 是否选中)	一个编辑

如果多选型可选性很多，比如有10000项，这个怎么处理？ Mask Color --多选
--10,20,30,40,50
--000000000
--1000000000 --10选中
--0100000000 --20选中

对于多选型参数有大量选项（如10000项）的情况，直接使用位掩码（bitmask）确实会受到整数位数的限制（32位或64位）。因此，我们需要采用其他方法来高效建模。以下是一种基于OR-Tools SAT的解决方案，使用多个整数变量（IntVar）作为位掩码段（bitmask segments）来扩展表示能力，并结合约束处理。同时，我会提供一个完整的代码示例和测试用例。

- **深入探索：**问题更加深入，能够触及技术本质

在配置器领域，都说基于约束求解器，没有方向性（先配置a，在配置b，在配置c，或先b，在c），那么基于规则引擎是否也是无方向性，请解释



☒ 已深度思考（用时 11 秒） ▾

• 好的，用户问的是在配置器领域，基于规则引擎是否和基于约束求解器一样没有方向性。首先得确认用户可能是在做技
简单直接的回答是：**不，基于规则引擎的配置在本质上是具有方向性的。**这正是它与基于约束求解器
(Constraint Solver) 的无方向性方法最根本的区别之一。

下面我将详细解释为什么，并对比两者的工作机理。

1. 基于约束求解器 (Constraint Solver)：真正的无方向性

工作原理：

约束求解器将整个配置问题建模为一个**约束满足问题 (CSP, Constraint Satisfaction Problem)**。它包含三个部分：

- **变量 (Variables)**：需要配置的选项（如：发动机型号、颜色、轮胎尺寸）。
- **值域 (Domains)**：每个变量可能的取值范围（如：颜色={红, 白, 蓝}）。
- **约束 (Constraints)**：变量之间必须满足的关系（如：IF 发动机=V8 THEN 变速箱=自动；豪华套装 REQUIRES 真皮座椅）。

无方向性的体现：

当用户做出一个选择时（例如，先选择“颜色=红色”），求解器并不是顺着一个预定的流程去检查后续选项。而是：

1. 将“颜色=红色”这个选择作为一个新的约束，添加到整个约束网络中。
2. 然后，**重新审视整个网络**，根据所有已知约束，**并行地、同时地**推导出所有其他变量的可能状态。

- **识别AI的局限性：** AI有时也会短路（或是触及到他的知识盲区），给的答案不合理的时候，这时候，还是回到解决问题最开始的地方。下面案例是大模型给出解决很多约束冲突的方法——松弛变量，但是给出代码的案例死活就是不对，给的推理过程很详实，但是逻辑就是不正确，尝试很多轮都回答不对，后来回到其源头：Or-tools源码，通过看源码及理解原理，后

来尝试出一个更简单、易懂的实现方案。

使用Ortools对一个配置问题进行建模，但是计算结果和预期的不同，怎么定位？请举例说明定位过程，主要使用最新的9.12.4544版本，不要使用老版本

3.3 约束松弛调试

当问题被判定为不可行时，逐步松弛约束以识别冲突：

python

复制 下载

问题描述

```
java

// 变量
x ∈ [0, 10], y ∈ [0, 10]

// 规则
规则1: x + y ≥ 15
规则2: x ≤ 5
规则3: y ≤ 8
```

1. 添加松弛变量

```
java

// 为每个规则添加松弛变量
松弛1 = 布尔变量(规则1可松弛)
松弛2 = 布尔变量(规则2可松弛)
松弛3 = 布尔变量(规则3可松弛)

// 带松弛的约束
x + y ≥ 15 OR 松弛1=true
x ≤ 5 OR 松弛2=true
y ≤ 8 OR 松弛3=true

// 目标: 最小化松弛的规则数量
最小化: 松弛1 + 松弛2 + 松弛3
```

2. 求解结果

```
text

最优解: x=7, y=8
松弛1=0 (规则1保持)
松弛2=1 (规则2松弛)
松弛3=0 (规则3保持)
目标值=1 (松弛1个规则)
```

怀疑addConditionalConstraintWithRelaxation方法实现有问题，运行上述程序，得到一个解是 x = 5
y = 0
z = 2，很明显这个解是不满足c3约束的，请修正

上述函数实现还是错的，运行上述代码，得到结果如下，很明显，结果是不满足约束2的，###运行结果：\n
求解状态: OPTIMAL
需要松弛的约束数量: 1

后期阶段：勤劳的AI高级程序员 VS 需求分析&测试人员，规范输出，保障质量

核心要点1：TDD驱动是保证质量的关键

随着项目框架已经构建，每次都是增量特性开发，也就是在已有的代码增加新功能，如何保证新增特性不影响老特性，这是工程化要解决的一个核心问题，引入大模型后，这个比传统人管理难度更大，大模型产生代码多且快，且没有关键代码和非代码区分，基本是一改一大片，默认生成的用例也很冗长，确认工作量大，TDD是关键，保证TDD高效落地要引导大模型生成高质量的精简的测试代码。

产品配置参数推理这个场景存在依赖基础数据构建复杂（简单Hello约150行书），输入和输出比较复杂，构建一个语句要200行代码，非常复杂。

```
82  @Test
83  public void testOnlyOneSolution() {
84      //构建基础数据
85      //.....
86      // 构建推理请求
87      InferParasReq req = new InferParasReq();
88      req.setModuleId(module.getId());
89      req.setEnumerateAllSolution(enumerateAllSolution: false); // 只返回一个解
90
91      // 设置主部件实例: tShirt11数量为1
92      PartInst mainPartInst = new PartInst();
93      mainPartInst.setCode(code: "tShirt11");
94      mainPartInst.setQuantity(quantity: 1);
95      req.setMainPartInst(mainPartInst);
96
97      // 执行参数推理
98      Result<List<ModuleInst>> result = ModuleConstraintExecutor.INST.inferParas(req);
99
100     // 验证结果
101     assertEquals(Result.SUCCESS, result.getCode(),
102         |         "Inference failed: " + result.getMessage());
103     assertNotNull(result.getData(), message: "Result data should not be null");
104     assertEquals(expected: 1, result.getData().size(), message: "Should have exactly one solution");
105
106     // 验证第一个解的参数值
107     ModuleInst solution = result.getData().get(index: 0);
108     assertNotNull(solution, message: "Solution should not be null");
109
110     // 验证color参数为Red
111     ParaInst colorPara = solution.getParas().stream()
112         |         .filter(p -> "color".equals(p.getCode()))
113         |         .findFirst()
114         |         .orElse(other: null);
115     assertNotNull(colorPara, message: "Color parameter should exist");
```

基于业务特点，使用大模型开发了测试框架。

通过业务注解，自动生成基础数据，5行注解->160行json数据



流式业务比较器，仅关注业务结果，3行代码1个用例，方便阅读

```
/**
 * 测试只有一个解的情况
 */
@Test
public void testOnlyOneSolution() {
    // 测试颜色参数推理
    inferParas(partCode: "tShirt11", qty: 1);

    // 验证第一个解
    solutions(index: 0).assertPara(code: "color").valueEqual(expectValue: "Red")
        | .assertPara(code: "size").valueEqual(expectValue: "Small");
    printSolutions();
}
```

测试框架引入，不仅保证了AI生成代码的质量，同时，也方便新人掌握这块代码，可为维护性大大提升。

核心要点2：高级程序员——及时重构

大家都知道，好代码逐步重构出来的，传统的代码重构工作量大，部分项目自动化测试体系也没有搭建起来，及时重构，成本高、质量风险大。

大模型引入了，他本身就是的代码高手，只有你需求（或方向）给得正确，就可以帮你高效完成。如：使用AI快速做重构—复杂逻辑抽取（人：提供重构线索）

- 重构前：

Prompt: "把选择的代码参考addCompatibleConstraintRequires泛化抽取为一个父类的函数Incompatible"

```

51 +     protected void addCompatibleConstraintIncompatible_note() {
52 +         // 创建条件变量: A在CA中
53 +         BoolVar inCA = model.newBoolVar("inCA");
54 +         model.addBoolOr(new Literal[]{
55 +             AVar.getParaOptionByCode("a1").getIsSelectedVar(),
56 +             AVar.getParaOptionByCode("a3").getIsSelectedVar()
57 +         }).onlyEnforceIf(inCA);
58 +         model.addBoolAnd(new Literal[]{
59 +             AVar.getParaOptionByCode("a1").getIsSelectedVar().not(),
60 +             AVar.getParaOptionByCode("a3").getIsSelectedVar().not()
61 +         }).onlyEnforceIf(inCA.not());
62 +
63 +         // 创建条件变量: B在CB中
64 +         BoolVar inCB = model.newBoolVar("inCB");
65 +         model.addBoolOr(new Literal[]{
66 +             BVar.getParaOptionByCode("b1").getIsSelectedVar(),
67 +             BVar.getParaOptionByCode("b2").getIsSelectedVar(),
68 +             BVar.getParaOptionByCode("b3").getIsSelectedVar()
69 +         }).onlyEnforceIf(inCB);
70 +         model.addBoolAnd(new Literal[]{
71 +             BVar.getParaOptionByCode("b1").getIsSelectedVar().not(),
72 +             BVar.getParaOptionByCode("b2").getIsSelectedVar().not(),
73 +             BVar.getParaOptionByCode("b3").getIsSelectedVar().not()
74 +         }).onlyEnforceIf(inCB.not());
75 +

```

- 重构后-父类新增函数:

```

src/main/java/com/jmix/configengine/artifact/ConstraintAlgImpl.java
387 + * 兼容性规则: Incompatible关系约束
388 + * 规则内容: 双向不兼容关系, 左侧参数和右侧参数不能在对应的组内同时存在
389 + * 例如: (a1,a3) Incompatible (b1,b2,b3) 表示:
390 + * - 如果A选择a1或a3, 则不能选择b1、b2或b3 (必须选择b4或b5)
391 + * - 如果A选择a2、a4或a5, 则可以选择任意值
392 + * - 如果B选择b1、b2或b3, 则A不能选择a1或a3 (必须选择a2、a4或a5)
393 + * - 如果B选择b4或b5, 则A可以选择任意值
394 + */
395 + public void addCompatibleConstraintIncompatible(String ruleCode, ParaVar leftParaVar, List<String> leftParaFilterOptionCodes,
396 +     ParaVar rightParaVar, List<String> rightParaFilterOptionCodes) {
397 +     // left: 确保只有一个参数选项被选中
398 +     model.addExactlyOne(leftParaVar.optionSelectVars().values().stream()
399 +         .map(option -> option.getIsSelectedVar())
400 +         .toArray(BoolVar[]::new));
401 +     // right: 确保只有一个参数选项被选中
402 +     model.addExactlyOne(rightParaVar.optionSelectVars().values().stream()
403 +         .map(option -> option.getIsSelectedVar())
404 +         .toArray(BoolVar[]::new));
405 +
406 +     // 由于左制右性, 左制右性由左小-右制译由

```

- 重构后-原有类做相应修改并补充测试用例:

```

src/test/java/com/jmix/configengine/scenario/ruletest/CompatibleRuleIncompatibleTest.java
29 + @ParaAnno(
30 +     options = {"b1", "b2", "b3", "b4", "b5"}
31 + )
32 + private ParaVar BVar;
33 +
34 + @Override
35 + protected void initConstraint() {
36 +     //注意: 一定要选择一个值怎么理解 (如果不选择, 也设置一个可选值就可以表达, 这样模型统一了)
37 +     //A=(a1,a2,a3,a4,a5), AVar.value的值域
38 +     //B=(b1,b2,b3,b4,b5), BVar.value的值域
39 +     // 规则内容: (a1,a3) Incompatible (b1,b2,b3), 则CA=(a1,a3), CB=(b1,b2,b3), NCA=(a2,a4,a5), NCB=(b4,b5)
40 +     // 解读:
41 +     // 1. 正向
42 +     // 1.1 如果AVar.value取CA的一个值(在CA中), 则BVar.value一定不是CB的一个值, 取CB内的值是非法的, 只能是NCB内的值, 如: 如果AVar.var=a1, 则BVar.var=b4或b5(NCB), 不可以是b1 或 b2 或 b3(CB)
43 +     // 1.2 如果AVar.value取NCA一个值(不在CA中), 则BVar.value可以是CB的一个值, 也可以是NCB的一个值, 如: 如果AVar.var=a2, 则BVar.var=b1或b2或b3(CB), 也可以是b4或b5(NCB)
44 +     // 2. 反向
45 +     // 2.1 如果BVar.value取CB的一个值(在CB中), 则AVar.value一定不是CA的一个值, 取CA内的值是非法的, 只能是NCA内的值, 如: 如果BVar.var=b1, 则AVar.var=a2或a4或a5(NCA), 不可以是a1 或 a3(CA)
46 +     // 2.2 如果BVar.value取NCB一个值(不在CB中), 则AVar.value可以是CA的一个值, 也可以是NCA的一个值, 如: 如果BVar.var=b4, 则AVar.var=a1 或 a3(CA), 也可以是a2 或 a4或a5(NCA)
47 +     // 使用泛化的Incompatible约束方法
48 +     addCompatibleConstraintIncompatible("rule1", AVar, Arrays.asList("a1", "a3"), BVar, Arrays.asList("b1", "b2", "b3"));
49 +
50 +

```

```

src/test/java/com/jmix/configengine/scenario/ruletest/CompatiableRuleIncompatibleTest.java
159 +
160 + @Test
161 + public void testInCAWithInCBIsInvalid() {
162 +     // 测试正向规则1.1: 如果A在CA中(a1), 则B在CB中(b1、b2或b3)是不合法的
163 +     inferParasByPara("A", "a1");
164 +     assertSolutionNum("B:b1", 0);
165 +     assertSolutionNum("B:b2", 0);
166 +     assertSolutionNum("B:b3", 0);
167 + }
168 +
169 + @Test
170 + public void testNotInCAAllowsAnyB() {
171 +     // 测试正向规则1.2: 如果A不在CA中(a2), 则B可以是CB或NCB中的任意值
172 +     inferParasByPara("A", "a2");
173 + }

```

核心要点2：高级程序员——新需求开发好手

为什么说他是“高级”，这里只需要提供他真正需求的规格就可以，不需要我们提供面面俱到的设计问题，及时口水稿就可以。

案例1：使用AI生成的自动测试函数

Prompt:

```

_**User (2025-09-06 09:32Z)**_

```

请按下列要求增强代码，实现方案如下：

1、在ConstraintAlgImpl增加方法

```
void autoAddConstraints(Module module) {
```

```
    对module.rules的每个rule
```

```
        根据rule.code(如:rule1)在当前类找到方法ruleMethod (如: protected void rule2() {})
```

```
        可调用ruleMethod, 完成添加和注册
```

```
}
```

2、在ConstraintAlgImpl.initConstraint调用autoAddConstraints

实现代码：

```
src/main/java/com/jmix/configengine/artifact/ConstraintAlgImpl.java
170 +  */
171 + protected void autoAddConstraints(Module module) {
172 +     if (module == null || module.getRules() == null || module.getRules().isEmpty()) {
173 +         log.debug("No rules found in module, skipping auto constraint registration");
174 +         return;
175 +     }
176 +
177 +     try {
178 +         // 获取当前类的所有方法
179 +         Method[] methods = this.getClass().getDeclaredMethods();
180 +         Map<String, Method> methodMap = new HashMap<>();
181 +
182 +         // 构建方法名到方法的映射，方便查找
183 +         for (Method method : methods) {
184 +             methodMap.put(method.getName(), method);
185 +         }
186 +
187 +         // 遍历所有规则
188 +         for (Rule rule : module.getRules()) {
189 +             if (rule == null || rule.getCode() == null || rule.getCode().trim().isEmpty()) {
190 +                 continue;
191 +             }
192 +
193 +             String ruleCode = rule.getCode();
194 +             // 构造方法名: addConstraint_ + ruleCode
195 +             String methodName = "" + ruleCode;
196 +
197 +             // 查找对应的方法
198 +             Method ruleMethod = methodMap.get(methodName);
199 +             if (ruleMethod != null) {
200 +                 try {
201 +                     // 设置方法可访问（处理protected方法）
202 +                     ruleMethod.setAccessible(true);
203 +                     // 调用方法
204 +                     ruleMethod.invoke(this);
205 +                     log.info("Successfully registered constraint for rule: {}", ruleCode);
206 +                 } catch (Exception e) {
207 +                     log.error("Failed to invoke rule method: {} for rule: {}", methodName, ruleCode, e);
208 +                 }
209 +             }
210 +         }
211 +     }
212 + }
```

案例2：修改编译错误

```
569 ---
570
571 现在我看到了真正的问题！错误信息显示在 `ModuleAlgArtifactGenerator.java` 的第167行有一个编译问题: `The method readValue(JsonPar
572
573 这说明 `ModuleAlgArtifactGenerator` 类中还在尝试将 `RuleSchema` 对象当作 JSON 来解析。让我查看这个类:
574
575
576 ---
804
805 太好了！现在测试全部通过了。让我总结一下我们所做的修改:
806
807 ## 修改总结
808
809 我们成功地将 `Rule` 类中的 `rawCode` 字段类型从 `String` 改为 `RuleSchema`，并同步修改了相关的测试代码和生成器代码。
810
811 ### 主要修改内容:
812
813 1. **修改了 `Rule.java` 模型类**:
814 | - 将 `rawCode` 字段类型从 `String` 改为 `RuleSchema`
815 | - 添加了必要的导入语句
816
817 2. **修改了 `ModuleAlgArtifactGeneratorTest.java` 测试类**:
818 | - 添加了必要的导入语句 (`RuleSchema`, `CompatibleRuleSchema`, `CalculateRuleSchema`, `SelectRuleSchema`, `ExprSchema`)
819 | - 修改了测试数据创建方法，使用具体的 `RuleSchema` 子类对象而不是 JSON 字符串
820 | - 为每个规则类型创建了相应的 Schema 对象，并设置了正确的属性
821
822 3. **修改了 `ModuleAlgArtifactGenerator.java` 生成器类**:
823 | - 修改了 `buildCompatibleRule` 方法，让它直接使用 `RuleSchema` 对象而不是尝试解析 JSON
824 | - 添加了类型检查，确保正确处理不同类型的规则
825
```

核心要点3：高级程序员——根据业务模型一步生成约束模式和测试代码

上面解释是就要Cursor等IDE工具，是类似程序员使用，这次验证，还基于大模型开发根据业务规范一步生成约束模型和测试代码生成器。

- 输入：业务模型及约束逻辑（自然语言，伪代码等）

```

J Hello2Gen.java  ≡ Hello2Spec.md ×  ≡ ConflictDebugSpec.md  J ConflictDebugGen.java

src > test > java > com > jmix > tool > modelgentest > ≡ Hello2Spec.md

1
2  ##userVariableModel
3  ```java
4  @ParaAnno(
5  |    options = {"Red", "Black", "White"}
6  )
7  private ParaVar colorVar;
8
9  @ParaAnno(
10 |    options = {"Small", "Medium", "Big"}
11 )
12 private ParaVar sizeVar;
13
14 @PartAnno()
15 private PartVar tShirt11Var;
16 ```
17
18 ##userLogicByPseudocode
19 ```java
20 | //rule1: 如果的颜色是红色且大小是小号是, 则tshirt11的数量为1, 否者为3
21 | ```
```

• 输出1: 基于约束模型的实现代码

```
bugIfThenTest.java | ConflictDebugIfThenSpec.md | ConflictDebugTest.java | Hello2Test.java | MyTShirtTest.java
src > test > java > com > jmix > tool > modelgentest > Hello2Test.java > Hello2Test > Hello2Constraint > rule1()

18 public class Hello2Test extends ModuleScenarioTestBase {
19
20     @ModuleAnno(id = 123L)
21     static public class Hello2Constraint extends ConstraintAlgImplTestBase {
22         @ParaAnno(options = { "Red", "Black", "White" })
23         private ParaVar colorVar;
24
25         @ParaAnno(options = { "Small", "Medium", "Big" })
26         private ParaVar sizeVar;
27
28         @PartAnno()
29         private PartVar tShirt11Var;
30
31         @CodeRuleAnno(normalNaturalCode = "如果的颜色是红色且大小是小号是, 则tShirt11的数量为1, 否者为3 ")
32         private void rule1() {
33             // Create condition variable: color is Red and size is Small
34             BoolVar redAndSmall = model.newBoolVar(name: "rule1_redAndSmall");
35
36             // Implement condition logic: redAndSmall = (color == Red) AND (size == Small)
37             model.addBoolAnd(new Literal[] {
38                 this.colorVar.getParaOptionByCode(code: "Red").getIsSelectedVar(),
39                 this.sizeVar.getParaOptionByCode(code: "Small").getIsSelectedVar()
40             }).onlyEnforceIf(redAndSmall);
41
42             // If not Red and Small combination, set redAndSmall to false
43             model.addBoolOr(new Literal[] {
44                 this.colorVar.getParaOptionByCode(code: "Red").getIsSelectedVar().not(),
45                 this.sizeVar.getParaOptionByCode(code: "Small").getIsSelectedVar().not()
46             }).onlyEnforceIf(redAndSmall.not());
47
48             // Set tShirt11 quantity based on condition
49             // If redAndSmall is true, set quantity to 1
50             model.addEquality(this.tShirt11Var.qty, value: 1).onlyEnforceIf(redAndSmall);
51
52             // If redAndSmall is false, set quantity to 3
53             model.addEquality(this.tShirt11Var.qty, value: 3).onlyEnforceIf(redAndSmall.not());
54         }
55     }
56 }
```

• 输出2: 对应的测试用例

```
OUTLINE | Hello2Test.java M
src > test > java > com > jmix > tool > modelgentest > Hello2Test.java > Hello2Test > Hello2Constraint > rule1()

18 public class Hello2Test extends ModuleScenarioTestBase {
19
20     public Hello2Test() {
21         super(constraintAlgClazz: Hello2Constraint.class);
22     }
23
24     @Test
25     public void testIfCondition() {
26         // Test when color is Red and size is Small, tShirt11 quantity should be 1
27         inferParas(partCode: "tShirt11", qty: 1);
28
29         resultAssert()
30             .assertSuccess()
31             .assertSolutionSizeEqual(expectedSize: 1);
32
33         solutions(index: 0).assertPara(code: "color").valueEqual(expectValue: "Red")
34             .assertPara(code: "size").valueEqual(expectValue: "Small");
35         printSolutions();
36     }
37 }
```


核心要点4：勤劳的程序员

让大模型修改的编程规范

```
7  /**User (2025-09-21 13:33Z)**_
8
9  检查所有的类（或接口），需要补齐类javadoc，包括功能说明，@since注解
10
11  例如：
12  /**
13   * 约束算法接口
14   *
15   * @since 2025-09-22
16   */
17  public interface ConstraintAlg {
18
19  }
20
21  ---
22
23
24  _**Assistant (default)**_
25
26  我来帮你检查所有类（或接口）并补齐Javadoc注释。首先让我了解项目结构，然后检查哪些类缺少Javadoc。
27
2059 <think>
2060 我已经完成了大部分的Javadoc补充工作。我可以看到用户要求检查所有的类（或接口），需要补齐类javadoc，包括功能说明，
2061
2062 让我总结一下我已经完成的工作：
2063 1. 扫描了所有Java类文件
2064 2. 识别了缺少Javadoc的类和接口
2065 3. 为大量重要的类添加了Javadoc注释，包括@since 2025-09-22注解
2066
2067 我已经为以下类别的类添加了Javadoc：
2068 - 核心模型类 (Module, Para, Part, Rule等)
2069 - 输出模型类 (ModuleInst, ParaInst, PartInst, Result等)
2070 - 算法模型类 (AlgCPModel, ParaVar, Var, PartVar等)
2071 - 工具类 (ModuleUtils, ModelHelper等)
2072 - 注解类 (ModuleAnno, ParaAnno等)
2073 - 实现类 (ModuleConstraintExecutorImpl等)
2074 - 制品生成器相关类 (VarInfo, ModuleVarInfo等)
2075 - 枚举类 (ParaType, PartType, ModuleType等)
2076 - 规则相关类 (RuleSchema, RuleTypeConstants等)
```

五、总结

在新技术项目中，AI可以显著提升开发效率，但必须明确人机协作的定位。人是决策者和质量把控者，AI是执行工具和知识助手。通过在不同阶段采用不同的协作策略，我们成功在1.5个月内完成了原本需要3-6个月的项目，证明了AI辅助开发在技术穿刺项目中的巨大价值。

针对约束求解本身的，下一步计划，应用场景广泛，在企业高频高能耗场景，是Agent的核心应用场景之一，如何重复大模型的自然语言理解、推理等能力（代码连接主义的）和类似约束求解器（代表符号主义）是解决企业复杂问题有效方法，下面根据Palantir画的架构图